



PRODI FISIKA
UNIVERSITAS BRAWIJAYA

Diseminasi Proyek Pemrograman Paralel I : Analisis Distribusi Waktu Peluruhan Rantai

AHMAD HANAFI HASAN | 245090300111001
STEPHEN TANGGUH LAIA | 245090300111010

LATAR BELAKANG

- Peluruhan radioaktif merupakan proses perubahan inti atom tidak stabil menjadi lebih stabil melalui emisi partikel atau radiasi.
- Rantai peluruhan Uranium-238 (U-238) penting dipelajari karena merupakan isotop uranium yang paling melimpah di alam dan meluruh hingga menjadi Timbal-206 (Pb-206).
- Pemodelan menggunakan Persamaan Bateman bersifat deterministik sehingga kurang menggambarkan sifat acak peluruhan radioaktif.
- Metode Monte Carlo mampu memodelkan peluruhan secara probabilistik, tetapi membutuhkan komputasi yang tinggi untuk rantai U-238.
- Oleh karena itu, penelitian ini mengembangkan simulasi peluruhan partikel paralel menggunakan metode Monte Carlo untuk meningkatkan efisiensi komputasi serta menganalisis distribusi waktu peluruhan dan karakteristik statistik setiap nuklida.

MODEL FISIKA

Hukum Peluruhan Radioaktif

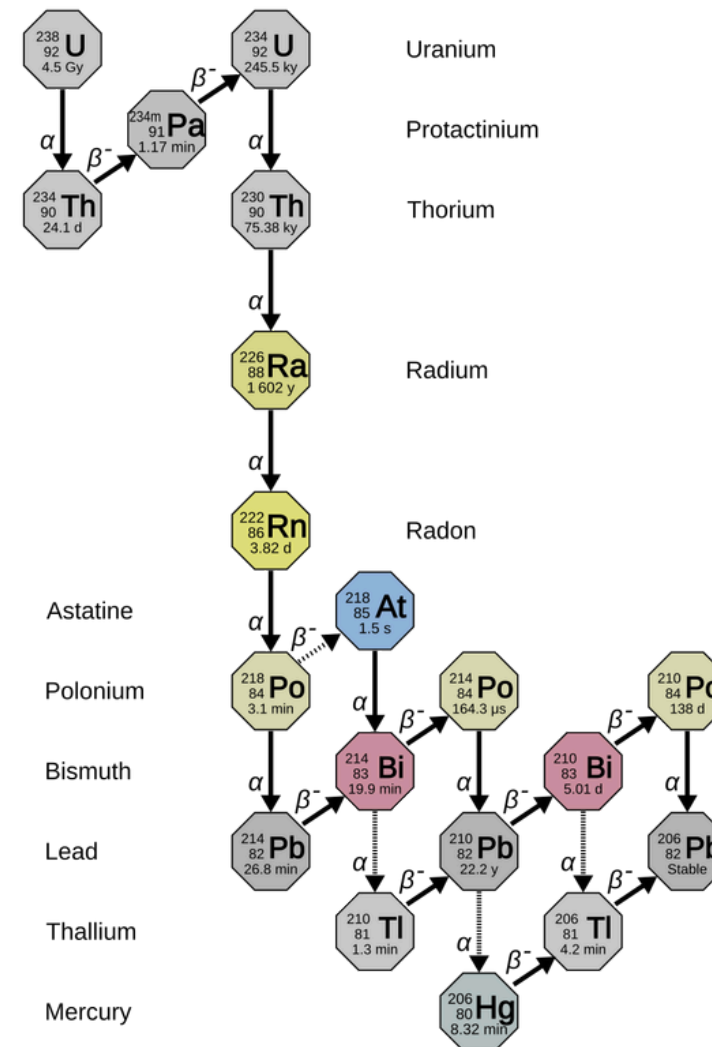
$$N(t) = N_0 e^{-\lambda t}$$

Distribusi Waktu

$$f(t) = \lambda e^{-\lambda t}$$

$$\tau = \frac{1}{\lambda} = \frac{T_{\frac{1}{2}}}{\ln 2}$$

MEKANISME PELURUHAN



Algoritma Monte Carlo

Untuk setiap event (satu atom U-238), algoritma bekerja sebagai berikut:

- Ambil 14 sampel acak dari distribusi eksponensial, masing-masing dengan mean lifetime τ_i isotop ke- i .
- Jumlahkan ke-14 waktu tunggu tersebut untuk mendapatkan total waktu peluruhan satu event.
- Ulangi untuk N event (misalnya 100.000 kali) untuk membangun distribusi statistik.

Secara matematis, total waktu peluruhan satu event adalah:

$$T_{\text{total}} = \sum_i t_i \quad \text{di mana } t_i \sim \text{Exp}(\tau_i), i = 1..14$$

Distribusi T_{total} didominasi oleh τ_1 (U-238 $\approx$ 6,45 miliar tahun), sehingga histogram berbentuk eksponensial yang sangat miring ke kanan.

MODEL FISIKA

Hukum Peluruhan Radioaktif

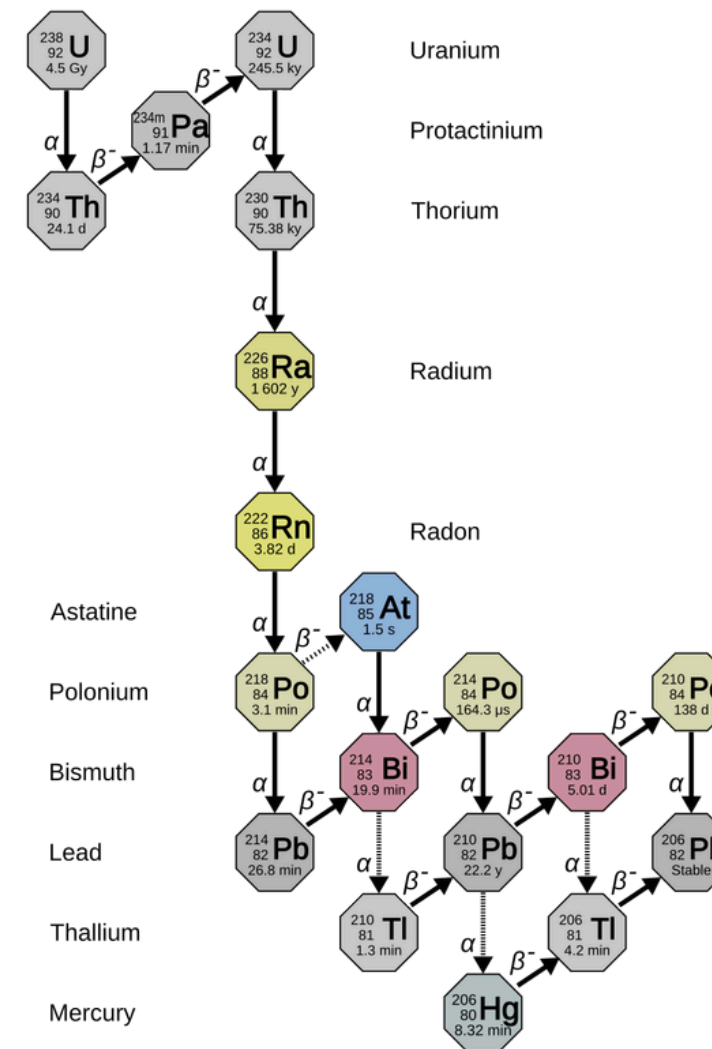
$$N(t) = N_0 e^{-\lambda t}$$

Distribusi Waktu

$$f(t) = \lambda e^{-\lambda t}$$

$$\tau = \frac{1}{\lambda} = \frac{T_{\frac{1}{2}}}{\ln 2}$$

MEKANISME PELURUHAN



Algoritma Monte Carlo

Untuk setiap event (satu atom U-238), algoritma bekerja sebagai berikut:

- Ambil 14 sampel acak dari distribusi eksponensial, masing-masing dengan mean lifetime τ_i isotop ke- i .
- Jumlahkan ke-14 waktu tunggu tersebut untuk mendapatkan total waktu peluruhan satu event.
- Ulangi untuk N event (misalnya 100.000 kali) untuk membangun distribusi statistik.

Secara matematis, total waktu peluruhan satu event adalah:

$$T_{\text{total}} = \sum_i t_i \quad \text{di mana } t_i \sim \text{Exp}(\tau_i), i = 1..14$$

Distribusi T_{total} didominasi oleh τ_1 (U-238 $\approx$ 6,45 miliar tahun), sehingga histogram berbentuk eksponensial yang sangat miring ke kanan.



STRATEGI PARALELISME

Bahasa dan Library yang digunakan

Komponen	Detail
Bahasa	Python 3
Library paralel	mpi4py — binding Python untuk MPI (Message Passing Interface)
Library numerik	NumPy
Visualisasi	Matplotlib (backend Agg — non-interactive)
Model paralel	SPMD (Single Program Multiple Data) via MPI
Pola komunikasi	Scatter beban + Gather hasil ke rank 0 (master)

Distribusi Beban Kerja (Data Parallelism)

```
# — Distribusi beban —————
# Rank 0 menghitung berapa event yang dikerjakan masing-masing rank.
base  = N_EVENTS // size
extra = N_EVENTS % size
# Rank 0..extra-1 masing-masing mengerjakan satu event lebih banyak
n_local = base + (1 if rank < extra else 0)
```

Independent Seeding per Proses

```
# — Seed independen per proses (SeedSequence menjamin bebas korelasi) —
seed_seq  = np.random.SeedSequence(MASTER_SEED)
child_seeds = seed_seq.spawn(size)
local_seed = int(child_seeds[rank].generate_state(1)[0])
```

Sinkronisasi dan Pengumpulan Hasil

```
# — Timer mulai (barrier memastikan semua proses mulai bersamaan) —
comm.Barrier()
t_start = MPI.Wtime()  # waktu wall-clock MPI (lebih presisi dari time.time)

# — Komputasi lokal —————
local_results = simulate_local(n_local, local_seed)

# — Kumpulkan hasil di rank 0 (Gather) —————
all_results = comm.gather(local_results, root=0)

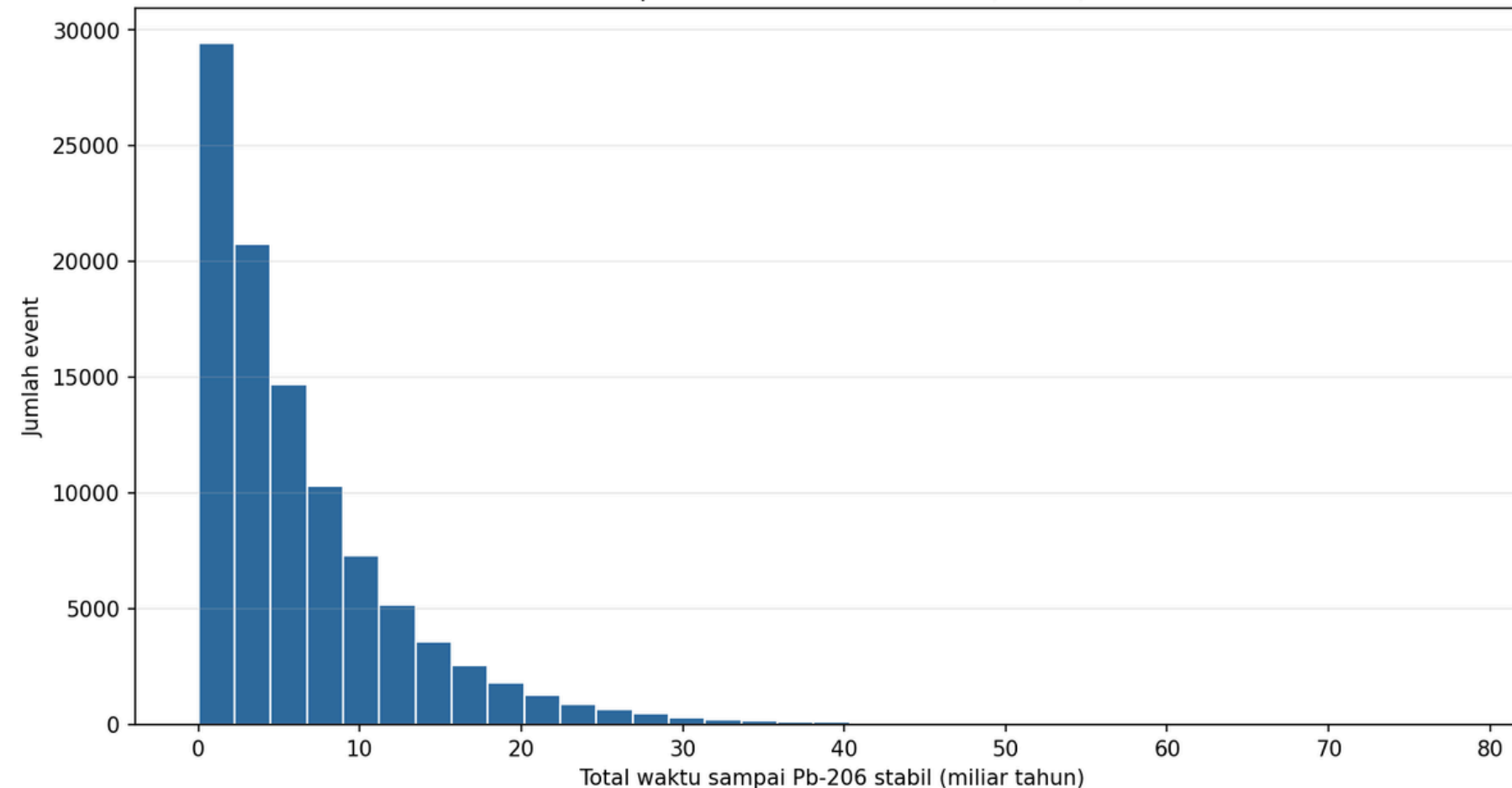
comm.Barrier()
t_end = MPI.Wtime()
```



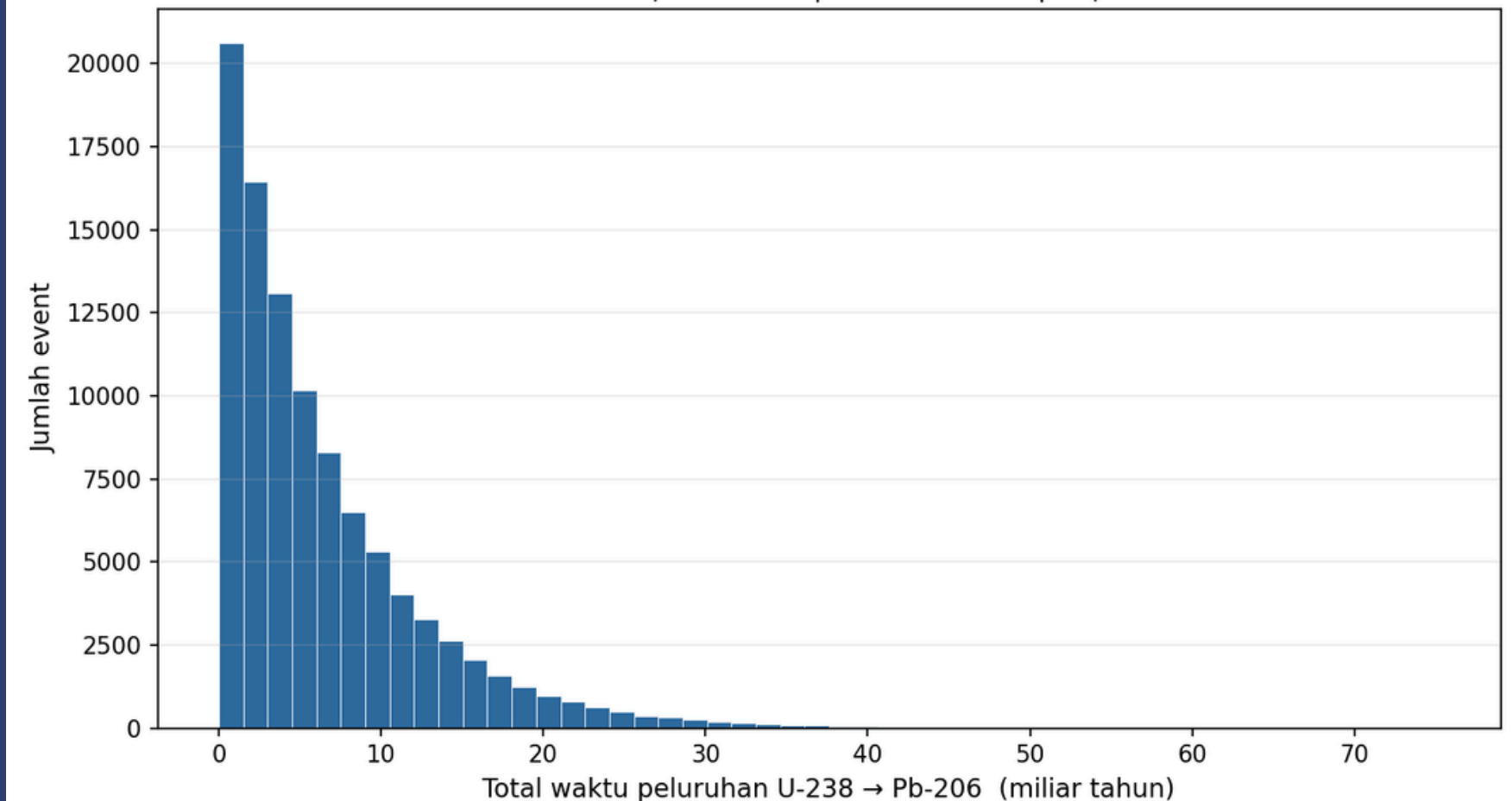
Kode Serial : Kode Serial Percobaan

Kode Paralel : Kode paralel percobaan

Distribusi waktu peluruhan U-238 ke Pb-206, serial, N=100.000



Distribusi waktu peluruhan | N = 100.000 | 3 proses MPI





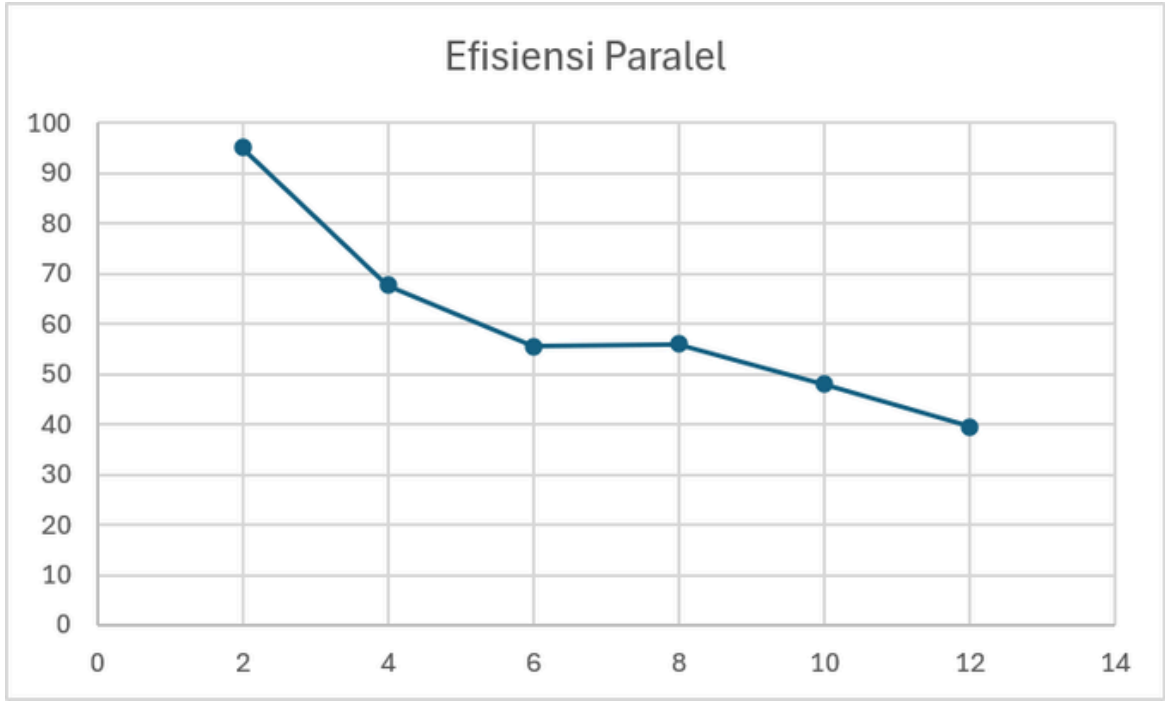
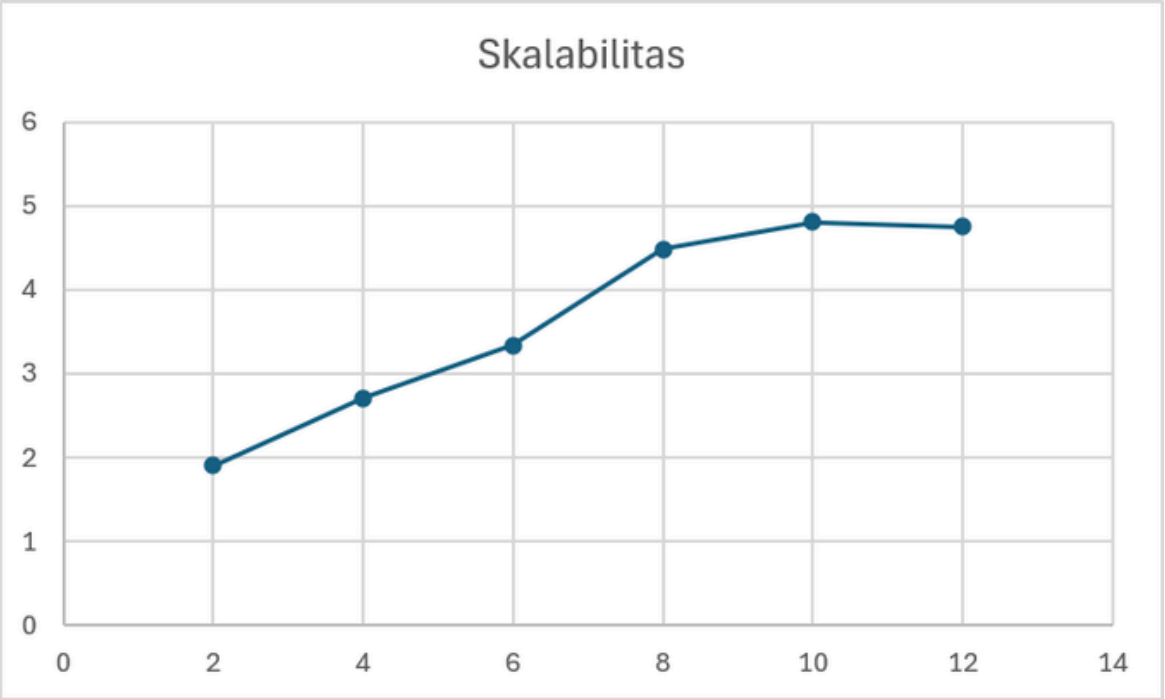
ANALISA PERFORMA

DATA WAKTU KOMPUTASI

Jumlah Prosesor/Thread	Waktu Komputasi (Detik)
Serial	0,052537
2	0,027617
4	0,019391
6	0,015765
8	0,011716
10	0,010938
12	0,011062

SPEED UP DAN EFISIENSI

Jumlah Prosesor/Thread	Speedup (S _p)	Efisiensi (%)
2	1,902	95,11%
4	2,709	67,73%
6	3,332	55,54%
8	4,484	56,05%
10	4,803	48,03%
12	4,749	39,58%



ANALISA PERFORMA

ANALISIS SPEED UP

- Paralelisasi MPI berhasil meningkatkan performa program.
- Peningkatan speedup tidak bersifat linear terhadap jumlah proses.
- Jumlah proses optimal pada pengujian ini adalah 10 proses MPI.
- Penambahan proses di atas nilai optimal menyebabkan keuntungan paralel berkurang akibat overhead komunikasi.

ANALISIS EFISIENSI

- Implementasi MPI berhasil meningkatkan performa simulasi.
- Efisiensi tertinggi diperoleh pada 2 proses (95,11%).
- Jumlah proses optimal berada di sekitar 10 proses.
- Overhead komunikasi dan sinkronisasi menjadi faktor pembatas utama pada jumlah proses yang lebih besar.
- Hasil pengujian sesuai dengan teori komputasi paralel dan Hukum Amdahl: penambahan prosesor tidak dapat meningkatkan performa secara tak terbatas.

ANALISA PERFORMA

ANALISIS KENDALA

Faktor utama yang membatasi skalabilitas:

1. Overhead komunikasi MPI Gather.
2. Ketidakseimbangan beban kerja.
3. Bagian program yang masih serial pada rank 0.
4. Keterbatasan perangkat keras.

Akibatnya, setelah 10 proses MPI, peningkatan performa menjadi sangat kecil dan cenderung mengalami penurunan.

SOLUSI YANG DISARANKAN

Optimasi dapat dilakukan melalui:

1. MPI Reduce untuk mengurangi komunikasi.
2. Dynamic Load Balancing untuk pemerataan beban kerja.
3. Parallel I/O atau distribusi pengolahan hasil untuk mengurangi bottleneck.
4. MPI + OpenMP sebagai komputasi hibrida.
5. GPU Computing (CUDA/OpenCL) untuk simulasi skala besar.

Pendekatan tersebut berpotensi memberikan peningkatan performa yang lebih signifikan dibandingkan hanya menambah jumlah proses MPI.

KESIMPULAN

Implementasi MPI berhasil meningkatkan performa simulasi dibandingkan eksekusi serial, ditunjukkan oleh peningkatan speedup dan penurunan waktu komputasi seiring bertambahnya jumlah proses. Kinerja optimal dicapai pada 10 proses dengan speedup maksimum 4,803. Namun, penambahan proses lebih lanjut tidak memberikan peningkatan yang signifikan karena adanya overhead komunikasi, sinkronisasi, ketidakseimbangan beban kerja, bagian program yang masih serial, serta keterbatasan perangkat keras. Secara keseluruhan, MPI menunjukkan skalabilitas yang baik hingga 10 proses, tetapi peningkatan performa selanjutnya memerlukan optimasi tambahan untuk mengurangi faktor-faktor pembatas tersebut.



PRODI FISIKA
UNIVERSITAS BRAWIJAYA

TANYA

JAWAB

Copyright © 2025 Hanafi Hasan



PRODI FISIKA
UNIVERSITAS BRAWIJAYA

**TERIMA
KASIH**

Copyright © 2025 Hanafi Hasan